

مفاهیم و فناوری‌های فراخوانی از راه دور (RPC)

اعضای تیم: سروش اصفهانیان 810101376 / نوید هاشمی 810101549

۱. تعریف (RPC (Remote Procedure Call

فراخوانی از راه دور یا RPC، نه تنها یک پروتکل، بلکه یک پارادایم ارتباطی در سیستم‌های توزیع شده است. این تکنولوژی بر اساس مدل کلاینت-سرور بنا شده و هدف آن فراهم کردن مکانیزمی است که به واسطه‌ی آن، یک پردازش (Process) در یک ماشین، بتواند کدی را در ماشین دیگر اجرا کند، به گونه‌ای که گویی یک فراخوانی محلی در فضای آدرس (Address Space) خودش رخ داده است.

از منظر فنی، RPC لایه‌ی Session (لایه ۵ مدل OSI) را مدیریت می‌کند تا پیچیدگی‌های لایه‌های پایین‌تر (مانند ایجاد اتصال TCP، مدیریت سوکت‌ها و کنترل جریان) را از لایه‌ی اپلیکیشن مخفی کند. در واقع RPC به برنامه‌نویس اجازه می‌دهد تا به جای درگیر شدن با `send()` و `receive()` در سطح بایت، با توابع سروکار داشته باشد.

۲. ایده اصلی فراخوانی تابع از راه دور

ایده اصلی RPC بر پایه‌ی شفافیت (Transparency) است، اما در سطوح حرفه‌ای، این شفافیت شامل موارد زیر می‌شود:

- **شفافیت نحوی (Syntactic Transparency):** فراخوانی یک تابع راه دور باید دقیقاً مشابه فراخوانی یک تابع محلی در کد منبع باشد.
- **پنهان سازی زیرساخت (Infrastructural Hiding):** برنامه‌نویس نباید درگیر تفاوت‌های سخت‌افزاری (مانند تفاوت در Endianness یا معماری ۶۴/۳۲ بیتی کلاینت و سرور) شود.
- **تغییر از Message Passing به Procedure Call:** در سیستم‌های قدیمی، برنامه‌ها با ارسال پیام‌های خام با هم حرف می‌زدند. RPC این پیام‌ها را به قالب آشناتر فراخوانی تابع درآورد تا انتزاع بالاتری ایجاد کند.

۳. تفاوت‌های فراخوانی محلی (LPC) و فراخوانی از راه دور (RPC)

اگرچه RPC تلاش می‌کند فرآیند فراخوانی را شفاف‌سازی کند، اما تفاوت‌های بنیادین میان محیط محلی و شبکه، چالش‌های پیچیده‌ای را ایجاد می‌کند. در فراخوانی محلی (LPC)، کلاینت و تابع فراخوانی شده در یک فضای آدرس مشترک قرار دارند، به این معنی که انتقال داده‌ها از طریق `stack` یا ثبات‌های پردازنده با سرعتی در مقیاس نانو ثانیه انجام می‌شود و امکان ارسال `pointer`‌ها برای دسترسی مستقیم به حافظه وجود دارد. اما در RPC، به دلیل جدایی کامل حافظه‌ها، مفهوم اشاره‌گر معنای خود را از دست می‌دهد و داده‌ها باید به صورت کامل کپی و ارسال شوند (اصطلاحاً به آن `Call-by-Copy` گفته می‌شود).

علاوه بر این، در محیط محلی، ما با یک مدل شکست واحد روبرو هستیم؛ یعنی یا کل برنامه اجرا می‌شود یا کل آن متوقف می‌گردد. در مقابل، RPC با پدیده‌ای به نام شکست جزئی (Partial Failure) روبروست که در آن ممکن است شبکه قطع شود یا پاسخ سرور در مسیر بازگشت گم شود، در حالی که هر دو ماشین به درستی در حال کار هستند. همچنین فاکتور زمان در RPC به دلیل تاخیرهای متغیر شبکه غیرقطعی است، در حالی که در LPC زمان اجرا کاملاً قابل پیش بینی و سریع است. همچنین تفاوت در نحوه نمایش داده‌ها (مانند ترتیب بایت‌ها یا Endianness) در ماشین‌های مختلف نیز لایه دیگری از پیچیدگی را به RPC اضافه می‌کند که در فراخوانی‌های محلی اساساً وجود ندارد.

خلاصه‌ی این تفاوت‌های کلیدی را می‌توان در جدول زیر مشاهده کرد:

ویژگی	فراخوانی محلی (LPC)	فراخوانی از راه دور (RPC)
اشتراک حافظه	کلاینت و سرور در یک حافظه مشترک هستند.	حافظه‌ها کاملاً مجزا هستند.
انتقال پارامتر	از طریق stack یا ارسال pointer.	فقط از طریق کپی داده‌ها.
مدل failure	شکست کامل یا موفقیت کامل.	امکان شکست جزئی (Partial Failure) شبکه.
زمان اجرا	Deterministic.	کندتر و تحت تاثیر latency شبکه.
نمایش داده‌ها	یکسان برای کلاینت و سرور.	احتمال تفاوت در معماری.

۴. بررسی نقش‌های Client Stub و Server Stub

در مدل RPC، برنامه‌های کلاینت و سرور نمی‌توانند به طور مستقیم با یکدیگر تبادل داده کنند، زیرا همانطور که اشاره شد ممکن است دو دنیای متفاوت باشند، یعنی در فضاهای آدرس مجزا قرار دارند و ممکن است با زبان‌های برنامه‌نویسی متفاوتی نوشته شده باشند. برای حل این شکاف، معماری RPC از دو موجودیت واسط به نام‌های Stub بهره می‌برد که در لایه اپلیکیشن عمل کرده و به عنوان یک لایه چسبنده بین برنامه و شبکه عمل می‌کنند.

۱. Client Stub (The Representative)

Client Stub در واقع یک پراکسی محلی است که در فضای آدرس کلاینت مستقر شده و وظیفه دارد شفافیت فراخوانی (Call Transparency) را در بالاترین سطح ممکن تامین کند. از منظر عملیاتی، زمانی که برنامه کلاینت یک تابع راه دور را صدا می‌زند، در حقیقت در حال تعامل با این stub است که signature تابعی دقیقاً مشابه تابع اصلی دارد. وظیفه محوری و پیچیده این لایه، فرآیند Marshalling است؛ به این معنا که stub باید آرگومان‌های دریافتی از stack را برداشته و آن‌ها را به یک فرمت داده‌ای استاندارد و مستقل از ماشین (Canonical Data Representation) تبدیل کند تا مشکلاتی نظیر تفاوت در ترتیب بایت‌ها (Endianness) یا تفاوت در معماری پردازنده‌ها (۳۲ یا ۶۴ بیتی) مانع از درک داده در مقصد نشود. پس از بسته بندی داده‌ها، client stub با کتابخانه زمان اجرای RPC

تعامل کرده، پیام را برای ارسال به لایه های پایین تر شبکه آماده می کند و تا زمان دریافت پاسخ از سرور، وضعیت اجرای کلاینت را در حالت blocking یا انتظار، مدیریت می نماید.

II. Server Stub (The Skeleton/Dispatcher)

Server Stub که در متون تخصصی اغلب از آن با عنوان Skeleton یا Dispatcher یاد می شود، نقش واسطه نهایی بین لایه شبکه و منطق برنامه در سمت سرور را ایفا می کند. این جزء وظیفه دارد بسته های بایت ارسالی از سوی کلاینت را دریافت کرده و عملیات معکوس Unmarshalling را روی آن ها اجرا کند؛ یعنی جریان بایت های خام را دوباره به ساختار های داده ای و اشیاء قابل فهم برای زبان برنامه نویسی سرور تبدیل نماید. پس از بازسازی داده ها، server stub با بررسی ID procedure تابع در هدر پیام، تشخیص می دهد که کدام متد یا تابع واقعی در لایه اپلیکیشن سرور باید فراخوانی شود. در ادامه پس از اجرای تابع توسط سرور و دریافت نتیجه، stub وظیفه دارد مقدار بازگشتی یا حتی پیام های خطای احتمالی را دوباره بسته بندی کرده و در قالب یک پیام پاسخ به سمت کلاینت ارسال کند تا چرخه فراخوانی تکمیل گردد.

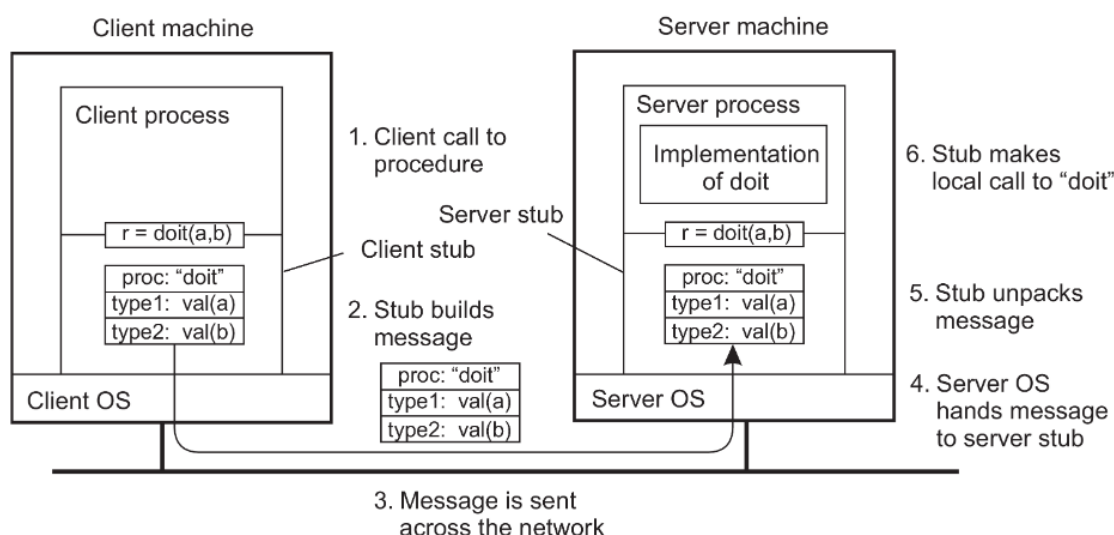


Figure 1 -نمایی ساده از یک نمونه فراخوانی از راه دور

۵. مفاهیم Deserialization و Serialization

در دنیای سیستم های توزیع شده، داده ها در حافظه RAM به صورت ساختار های غیرخطی و پیچیده (مانند درخت ها، گراف ها یا اشیاء با اشاره گر های متعدد) ذخیره می شوند که مستقیماً قابل انتقال روی سیم های شبکه نیستند. فرآیند Serialization (که در ادبیات RPC اغلب با Marshaling مترادف فرض می شود) به معنای تبدیل این ساختار های حافظه ای به یک جریان خطی از بایت ها (Byte Stream) است که بتواند بدون تغییر در طول شبکه منتقل شود. این فرآیند بسیار حساس است، زیرا باید تمام اطلاعات مربوط به نوع داده و ساختار آن را حفظ کند تا در مقصد قابل بازسازی باشد. در نقطه مقابل، Deserialization فرآیندی است که در آن، گیرنده جریان بایت های خام را دریافت کرده و با استفاده از قواعد از پیش تعیین شده، آن ها را دوباره به اشیاء و ساختار های داده ای در فضای آدرس خود تبدیل می کند. تفاوت کلیدی در پروتکل های مدرن RPC (مانند gRPC) در بهینگی این فرآیند است؛ به طوری که Binary Serialization به دلیل حجم کمتر و سرعت

پردازش بالاتر، جایگزین فرمت های متنی مانند XML شده است تا سربرار پردازنده در سیستم های پرتراфик به حداقل برسد.

۶. مفهوم (Interface Definition Language) IDL

یکی از چالش های بزرگ در سیستم های توزیع شده، ناهماهنگی میان زبان های برنامه نویسی و معماری های مختلف کلاینت و سرور است. IDL به عنوان یک زبان میانی و مستقل (Language-Agnostic) عمل می کند که وظیفه آن تعریف قرارداد بین کلاینت و سرور است. در واقع، برنامه نویس به جای نوشتن مستقیم کد ارتباطی، ابتدا امضای توابع، نام متد ها و نوع پارامتر های ورودی و خروجی را در یک فایل با نحو IDL تعریف می کند. این فایل سپس توسط یک کامپایلر مخصوص (مانند protoc در gRPC) پردازش شده و کد های stub مورد نیاز را به زبان های تولید می کند. اهمیت IDL در این است که به توسعه دهنده اجازه می دهد بدون نگرانی از جزئیات پیاده سازی زبان مقصد، بر روی منطق توزیع شده تمرکز کند و تضمین نماید که هر دو سمت ارتباط، از یک ساختار داده ای واحد و توافق شده پیروی می کنند. بدون وجود IDL، مدیریت تغییرات در سیستم های بزرگ که شامل صد ها سرویس مختلف هستند، عملاً غیرممکن خواهد بود.

۷. خطاهای رایج و معنانشناسی شکست در RPC

در سیستم های توزیع شده، Failure یک وضعیت استثنایی نیست، بلکه بخشی از ماهیت سیستم است. برخلاف یک برنامه محلی که در آن اگر کلاینت یا تابع دچار خطا شوند، کل فرآیند متوقف می شود، در RPC ما با پدیده Partial Failure روبرو هستیم؛ وضعیتی که در آن بخشی از سیستم (مثلاً شبکه یا یکی از ماشین ها) دچار اختلال شده در حالی که سایر بخش ها به درستی در حال کار هستند. این عدم قطعیت باعث می شود که کلاینت نتواند به راحتی تشخیص دهد که آیا درخواستش به مقصد نرسیده، یا سرور پس از انجام کار کرش کرده و نتوانسته پاسخ را بفرستد. برای درک بهتر این چالش ها، موارد زیر را به عنوان سناریو های شکست فنی بررسی می کنیم:

۱. خطای مکان یابی یا اتصال (Unable to Locate Server):

این خطا زمانی رخ می دهد که کلاینت نمی تواند سرور مورد نظر را در شبکه پیدا کند. این اتفاق ممکن است به دلیل خاموش بودن ماشین سرور، تغییر آدرس IP بدون اطلاع کلاینت، یا خرابی در سرویس دایرکتوری (Name Service) که وظیفه نگاشت نام ها به آدرس های شبکه را دارد، رخ دهد. در چنین حالتی، client stub معمولاً با یک پیام خطا یا استثنا مواجه می شود که نشان دهنده عدم برقراری اتصال اولیه است. از منظر فنی، حل این مشکل نیازمند مکانیزم های Dynamic Binding یا استفاده از Load Balancer ها است تا کلاینت همیشه به یک نقطه معتبر متصل شود.

۲. گم شدن پیام درخواست (Lost Request Message):

در این سناریو، پیام حاوی فراخوانی تابع به دلیل شلوغی شبکه یا خطای روتر ها، هرگز به سرور نمی رسد. این ساده ترین نوع خطا برای مدیریت است؛ کلاینت پس از گذشت یک بازه زمانی مشخص (Timeout)، متوجه عدم دریافت پاسخ شده و فرض می کند درخواستش گم شده است. راهکار رایج در اینجا ارسال مجدد است. اما چالش فنی زمانی آغاز می شود که کلاینت نمی داند پیام واقعاً گم شده یا فقط در شبکه با تاخیر مواجه شده است، که در صورت ارسال مجدد غیرضروری، ممکن است بار شبکه بیهوده افزایش یابد.

۳. توقف سرور پس از دریافت درخواست (Server Crash):

این یکی از پیچیده ترین خطاها در سیستم های توزیع شده است. تصور کنید درخواست به سرور می رسد، سرور شروع به پردازش می کند (مثلاً کسر مبلغ از یک حساب بانکی)، اما دقیقاً قبل از ارسال پاسخ به کلاینت، سرور کرش می کند یا برق ماشین سرور قطع می شود. وقتی کلاینت Timeout می دهد و دوباره درخواست را می فرستد، اگر سرور بازیابی شده باشد، ممکن است عملیات را دوباره انجام دهد که منجر به خطای منطقی (مثلاً کسر دوبرابری پول) می شود. برای حل این مشکل، مفهوم Idempotency (توان دستی) مطرح می شود؛ یعنی توابع باید به گونه ای طراحی شوند که اجرای چندین باره آن ها، تاثیری مشابه یک بار اجرا داشته باشد.

۴. گم شدن پیام پاسخ (Lost Reply Message):

در این حالت، سرور با موفقیت تابع را اجرا کرده و پاسخ را ارسال نموده است، اما پیام پاسخ در مسیر بازگشت به کلاینت گم می شود. کلاینت که پاسخی دریافت نکرده، طبق پروتکل خود Timeout می دهد. مشکل اینجا است که کلاینت نمی داند آیا سرور اصلاً درخواست را نگرفته، یا گرفته و انجام داده اما پاسخ گم شده است. اگر عملیات حساس باشد (مانند ثبت یک سفارش)، تکرار درخواست توسط کلاینت می تواند منجر به ثبت سفارش تکراری شود. در سیستم های پیشرفته، از Sequence Numbers ها برای هر درخواست استفاده می شود تا سرور بفهمد این یک درخواست تکراری برای کاری است که قبلاً انجام داده و فقط باید آخرین پاسخ را دوباره ارسال کند.

۵. توقف کلاینت پس از ارسال درخواست (Client Crash):

گاهی اوقات کلاینت درخواستی را می فرستد و در حالی که سرور مشغول پردازش سنگین روی آن است، کلاینت کرش می کند یا اتصالش را از دست می دهد. در این حالت سرور در حال مصرف منابع برای پاسخی است که هیچگاه دریافت نخواهد شد. به این وضعیت در متون تخصصی Orphan Call (فراخوانی یتیم) گفته می شود. برای مدیریت این وضعیت، تکنیک هایی مانند Extermination (کشتن پردازش های یتیم توسط کلاینت پس از راه اندازی مجدد) یا Reincarnation (تقسیم بندی زمان به دوره های مختلف و ابطال درخواست های دوره های قبلی) استفاده می شود تا منابع سرور هدر نرود.

۸. مقایسه REST با RPC

- تعریف و ماهیت REST

معماری REST اولین بار توسط روی فیلدینگ در سال ۲۰۰۰ معرفی شد و هدف آن بهره گیری حداکثری از ویژگی های پروتکل HTTP بود. در این مدل، هر موجودیت در سیستم (مانند یک کاربر، یک فایل یا یک تراکنش) به عنوان یک منبع در نظر گرفته می شود که از طریق یک URI منحصر به فرد (مانند users/123) قابل دسترسی است. برخلاف RPC که در آن ما نگران چگونگی انجام کار و نام توابع هستیم، در REST تمرکز بر روی وضعیت منبع است. تعامل با این منابع از طریق متدهای استاندارد HTTP شامل GET (خواندن)، POST (ایجاد)، PUT (به روز رسانی کل) و DELETE (حذف) انجام می شود. یکی از ویژگی های بنیادی REST، Statelessness بودن آن است؛ به این معنا که هر درخواست کلاینت به سرور باید تمام اطلاعات لازم برای درک و پردازش آن درخواست را همراه داشته باشد و سرور هیچ Context از درخواست های قبلی را در حافظه

خود نگه نمی‌دارد. این ویژگی باعث می‌شود که مقیاس‌پذیری در سیستم‌های REST بسیار بالا باشد، زیرا هر سروری در یک کلاستر می‌تواند به هر درخواستی پاسخ دهد.

-مقایسه با RPC

تفاوت بنیادین این دو در نگرش به ارتباطات است. در RPC، کلاینت احساس می‌کند که در حال کدنویسی محلی است و می‌خواهد یک عمل را انجام دهد (مثلاً `transferMoney()`)؛ اما در REST، کلاینت با یک اسم تعامل دارد و می‌خواهد وضعیت آن را تغییر دهد. از نظر Coupling، سیستم‌های RPC معمولاً Coupling بالاتری دارند، زیرا کلاینت باید دقیقاً از امضای تابع و پارامترهای سرور آگاه باشد (که توسط IDL تعیین می‌شود). در مقابل، REST به دلیل استفاده از یک Uniform Interface و متدهای استاندارد، Coupling کمتری دارد و تغییر در پیاده‌سازی سرور اغلب تأثیر کمتری بر کلاینت می‌گذارد.

از منظر کارایی و سربر، پروتکل‌های مدرن RPC (مانند gRPC که از HTTP/2 و پروتکل باینری استفاده می‌کنند) بسیار سریع‌تر از REST هستند. REST معمولاً از فرمت JSON استفاده می‌کند که مبتنی بر متن است و حجم بیشتری اشغال می‌کند و فرآیند Parse کردن آن توسط پردازنده، سنگین‌تر است. همچنین، RPC برای سناریوهایی که نیاز به محاسبات سنگین و تعاملات سریع بین میکروسرویس‌ها در یک شبکه داخلی دارند، گزینه بهینه‌تری محسوب می‌شود. در حالی که REST به دلیل سادگی و پشتیبانی گسترده توسط مرورگرها، پادشاه بی‌رقیب API‌های عمومی و وب‌سرویس‌هایی است که قرار است توسط توسعه‌دهندگان مختلف در بسترهای متفاوت استفاده شوند.

در بحث مدیریت خطا و Caching، REST به دلیل وفاداری به پروتکل HTTP، می‌تواند به خوبی از مکانیزم‌های Caching وب و کدهای وضعیت استاندارد (مانند 404 برای یافت نشدن منبع یا 500 برای خطای سرور) استفاده کند. در مقابل، در RPC اغلب درخواست‌ها با متد POST ارسال می‌شوند و حتی اگر خطایی در لایه اپلیکیشن رخ دهد، کد وضعیت HTTP ممکن است همچنان 200 (Success) باشد و جزئیات خطا در بدنه پیام مخفی شود. این موضوع دیباگ کردن سیستم‌های RPC را در لایه‌های شبکه پیچیده‌تر می‌کند.

۹. مقایسه چند فناوری رایج RPC

در طراحی سیستم‌های توزیع شده، انتخاب فناوری RPC بر اساس نیازهای معماری، کارایی و محدودیت‌های پهنای باند صورت می‌گیرد. هر یک از فناوری‌ها، لایه متفاوتی از انتزاع و کارایی را ارائه می‌دهند که در ادامه در ابتدا یک تحلیل توضیحی کوتاه از چهار مورد از برجسته‌ترین آن‌ها پرداخته می‌شود و سپس به مقایسه آنها می‌پردازیم:

- فناوری (Google Remote Procedure Call) gRPC :

این فناوری به عنوان یک استاندارد مدرن در معماری‌های میکروسرویس شناخته می‌شود. gRPC بر پایه پروتکل انتقال HTTP/2 بنا شده است که با استفاده از قابلیت Multiplexing، امکان ارسال چندین درخواست همزمان روی یک اتصال TCP واحد را فراهم می‌کند. از نظر فنی، تمایز اصلی gRPC در بهره‌گیری از Protocol Buffers به عنوان مکانیزم سریال‌سازی است. پروتوباف یک روش سریال‌سازی باینری است که داده‌ها را به شکلی بسیار فشرده کدگذاری کرده و سرعت پردازش آن به دلیل عدم نیاز به پارس کردن متنی، بسیار فراتر از JSON و XML است. همچنین، gRPC با پشتیبانی بومی از

Streaming دوطرفه، اجازه می‌دهد که کلاینت و سرور به صورت همزمان جریان های داده را ارسال و دریافت کنند، که این امر برای سیستم های مانیتورینگ و بلادرنگ بسیار حیاتی است.

- فناوری JSON-RPC و سادگی ساختاری:

این پروتکل با هدف ارائه یک راهکار سبک و Stateless طراحی شده است. JSON-RPC از فرمت متنی JSON برای کدگذاری پیام ها استفاده می‌کند که علی‌رغم افزایش حجم داده نسبت به حالت باینری، به دلیل خوانایی بالا توسط انسان ، فرآیند اشکال زدایی و مانیتورینگ شبکه را تسهیل می‌کند. این پروتکل وابسته به لایه انتقال خاصی نیست و می‌تواند بر روی HTTP یا TCP پیاده‌سازی شود. بدنه پیام در JSON-RPC شامل یک ساختار ساده برای مشخص کردن نام متد و پارامتر هاست. همچنین فقدان یک لایه اجباری برای تعریف واسط (IDL) ، پیاده‌سازی آن را در سیستم های کوچک بسیار سریع می‌کند، اما در سیستم های بزرگ توزیع شده، مدیریت تغییرات در امضای توابع را به دلیل نبود قرارداد سخت گیرانه، دشوار می‌سازد.

- فناوری Apache Thrift و انعطاف‌پذیری معماری:

Thrift که در ابتدا توسط فیس‌بوک توسعه یافت، یک چارچوب RPC با معماری چند لایه است که اجازه می‌دهد لایه های سریال‌سازی و انتقال به صورت مستقل انتخاب شوند. پشته تکنولوژی Thrift شامل لایه های پروتکل (فرمت داده) و transport (نحوه ارسال بایت ها) است. برخلاف gRPC که محدود به HTTP/2 است، Thrift می‌تواند بر روی انواع سوکت های Blocking و Non-blocking اجرا شود. زبان تعریف واسط (IDL) در Thrift بسیار قدرتمند است و امکان تولید کد برای طیف گسترده‌ای از زبان های برنامه‌نویسی را فراهم می‌کند. این فناوری برای سیستم هایی که نیاز به کارایی باینری در کنار انعطاف پذیری در پروتکل های انتقال دارند، گزینه‌ای ایده‌آل محسوب می‌شود.

- فناوری XML-RPC و میراث وب:

به عنوان یکی از پیشگامان پروتکل های RPC در بستر وب، XML-RPC از XML برای بسته بندی داده ها و از HTTP به عنوان پروتکل انتقال بهره می‌برد. از منظر مهندسی، این پروتکل به دلیل ساختار درختی و کلمات توصیفی XML ، دارای بیشترین میزان سربار در بین تمامی فناوری های یادشده است. فرآیند Marshaling و Unmarshaling در XML-RPC به دلیل ماهیت متنی و پیچیدگی پارس کردن تگ های XML، منابع پردازشی زیادی را مصرف می‌کند. امروزه این فناوری بطور عمده در سیستم های قدیمی یا سناریو هایی که سازگاری با زیرساخت های Legacy ضرورت دارد، مورد استفاده قرار می‌گیرد.

اکنون با توجه به مواردی که مطرح شد، این چهار فناوری RPC بر اساس شاخص های کلیدی زیر قابل طبقه‌بندی و مقایسه هستند. این مقایسه بر پایه هشت محور اصلی شامل ساختار داده، روش تعریف واسط، پشتیبانی از زبان ها، کارایی، سادگی پیاده‌سازی، تناسب با سیستم های توزیع شده، قابلیت Streaming و موارد کاربرد تدوین شده است.

جدول ۱: مقایسه تطبیقی فناوری های RPC بر اساس معیارهای هشت‌گانه

محور مقایسه	gRPC	JSON-RPC	Apache Thrift	XML-RPC
نوع داده و فرمت پیام	Binary (Protobuf)	Text (JSON)	Binary / Compact	Text (XML)

روش تعریف interface	IDL (proto file)	Implicit	IDL (thrift file)	Implicit
پشتیبانی از زبان ها	Go, C++, Java, Python, C#, Node.js, Dart	Go, JavaScript, Python, Java, Ruby (تقریباً همه زبان ها)	Go, C++, Java, Python, PHP, Erlang, Rust	Python, Java, PHP, Perl, C#
کارایی و سربار ارتباطی	بسیار بالا (سربار اندک)	متوسط	بالا	پایین (سربار زیاد)
سادگی پیاده سازی	متوسط	بسیار ساده	پیچیده	ساده
تناسب با سیستم های توزیع شده	عالی (میکروسرویس)	مناسب برای سیستم های کوچک	عالی (سیستم های وسیع)	محدود به سیستم های قدیمی
پشتیبانی از Streaming	کامل (دوطرفه)	ندارد	محدود	ندارد
موارد رایج استفاده	ارتباطات حساس به سرعت	API های وب و داخلی	سیستم های چندزبانه بزرگ	یکپارچه سازی های قدیمی

ذکر این نکته نیز مفید است که در فناوری‌هایی مانند gRPC و Apache Thrift، پشتیبانی از زبان ها از طریق تولید خودکار کد انجام می‌شود. به این معنا که با تعریف یک فایل واسط (IDL)، کامپایلر مخصوص آن فناوری، کد های لازم برای کلاینت و سرور را به زبان های هدف (مانند Go یا ++C) تولید می‌کند که این امر باعث هماهنگی کامل بین سرویس هایی می‌شود که با زبان های مختلف نوشته شده‌اند.

در مقابل، JSON-RPC و XML-RPC به دلیل تکیه بر فرمت های متنی استاندارد، نیازی به تولید کد ندارند و هر زبانی که قابلیت ارسال درخواست‌های HTTP و پارس کردن متون JSON یا XML را داشته باشد، می‌تواند به راحتی به عنوان کلاینت یا سرور در این پروتکل‌ها عمل کند.

علاوه بر شاخص های فنی، انتخاب یک فناوری در پروژه های واقعی سیستم های توزیع شده مستلزم تطبیق ویژگی های آن با نیاز پروژه است. جدول زیر راهنمایی برای این انتخاب بر اساس اولویت های مختلف ارائه می‌دهد:

جدول ۲: تحلیل فناوری بر اساس سناریوهای عملیاتی (Use Cases)

اولویت اصلی پروژه	فناوری پیشنهادی	دلیل انتخاب
بیشترین کارایی و سرعت	gRPC	استفاده از HTTP/2 و سریال سازی باینری جهت کاهش تأخیر.
سرعت در توسعه و سادگی	JSON-RPC	عدم نیاز به فایل‌های واسط (IDL) و استفاده از فرمت متنی رایج.
انعطاف در لایه انتقال	Apache Thrift	امکان تغییر پروتکل از باینری به متنی بدون تغییر در منطق برنامه.

پرسش های تحلیلی اجباری

1 چرا RPC باعث می شود فراخوانی یک سرویس راه دور شبیه فراخوانی یک تابع محلی بنظر برسد؟

دلیل اصلی اینکه RPC می تواند فراخوانی یک سرویس از راه دور را مشابه یک تابع محلی جلوه دهد، بهره گیری از شیوه انتزاع از طریق پراکسی است. این فرآیند از طریق چهار لایه کلیدی زیر محقق می شود:

- استفاده از Stub ها به عنوان جانشین:

در مدل RPC، برنامه نویس مستقیما با کد های شبکه درگیر نمی شود. به جای آن، از یک شیء محلی به نام Client Stub استفاده می کند که دقیقا همان امضای تابع مورد نظر را دارد. وقتی برنامه نویس تابع را فراخوانی می کند، در واقع در حال صدا کردن یک تابع محلی در حافظه کلاینت است؛ بنابراین از دید کدنویس، تفاوتی بین این فراخوانی و یک تابع معمولی وجود ندارد.

- پنهان سازی فرآیند Marshalling:

تمام پیچیدگی های تبدیل داده های انتزاعی به بایت های قابل انتقال در شبکه (که تحت عنوان Serialization توضیح داده شد) و برعکس، درون Stub ها مخفی شده است. پارامتر های تابع توسط Stub بسته بندی شده و به لایه شبکه تحویل داده می شوند، بدون اینکه بدنه اصلی برنامه از این فعل و انفعالات باخبر شود.

- مدیریت stack مشابه:

در فراخوانی محلی، پارامتر ها روی Stack قرار می گیرند تا تابع آن ها را بردارد. در RPC، سیستم این رفتار را شبیه سازی می کند؛ یعنی کلاینت پارامتر ها را به Stub می دهد و در سمت دیگر، Server Stub این پارامتر ها را از بسته شبکه استخراج کرده و دقیقا مانند یک فراخوانی محلی، آن ها را به تابع اصلی در سرور پاس می دهد.

- پنهان سازی لایه انتقال (Transport Layer):

برنامه نویس در لایه اپلیکیشن، درگیر مفاهیمی مثل ایجاد سوکت TCP/IP، مدیریت بسته های گم شده یا ترتیب بایت ها نمی شود. لایه RPC این اطمینان را ایجاد می کند که داده ها به مقصد می رسند و پاسخ بازگردانده می شود، به طوری که برای برنامه اصلی، این تاخیر زمانی شبکه، صرفا مانند یک محاسبه طولانی در یک تابع محلی به نظر می رسد.

همه این موارد به نوعی بیان می کنند که RPC با ایجاد یک توهم هم مکانی، پیچیدگی های توزیع شدگی را به زیر لایه ی فراخوانی تابع منتقل می کند تا تمرکز برنامه نویس صرفا بر منطق برنامه باقی بماند.

2 چرا این شباهت می تواند خطرناک یا گمراه کننده باشد؟

شباهت ظاهری RPC به فراخوانی های محلی، اگرچه یک انتزاع قدرتمند برای ساده سازی فرآیند توسعه محسوب می شود، اما در عین حال می تواند به عنوان یک شمشیر دولبه عمل کند و برنامه نویس را دچار خطای شناختی Fallacies of Distributed Computing - فرضیات نادرستی مانند قابل اطمینان بودن شبکه، صفر بودن تأخیر یا بی نهایت بودن پهنای باند که توسعه دهندگان سیستم های سنتی اغلب به طور ناخودآگاه آن ها را حقیقی پنداشته و به محیط ناپایدار شبکه تعمیم می دهند- کند. اصلی ترین و خطرناک ترین جنبه این

مسئله، تفاوت بنیادین در مدل شکست است. همانطور که در بخش مقایسه RPC و LPC گفته شد، در فراخوانی های محلی، برنامه یا با موفقیت اجرا می شود یا در صورت بروز مشکل، کل پردازش متوقف می گردد و سیستم به وضعیت قابل پیش بینی باز می گردد، اما در RPC ما با Partial Failure روبرو هستیم. در این حالت ممکن است بخشی از شبکه قطع شود، سرور پس از دریافت درخواست کرش کند، یا پاسخ در مسیر بازگشت گم شود، در حالی که کلاینت همچنان در حال اجراست. اگر برنامه نویس با تصور اینکه فراخوانی راه دور مشابه یک فراخوانی محلی است کدنویسی کند، ممکن است نسبت به مدیریت این موقعیت های مبهم غافل بماند و سیستم را در یک وضعیت ناسازگار یا داده های تکراری و ناهماهنگ رها کند.

علاوه بر این، تفاوت فاحش در میزان تأخیر نیز یکی دیگر از دام های این شباهت است. فراخوانی های محلی در مقیاس نانو ثانیه یا میکرو ثانیه انجام می شوند و حافظه اصلی در دسترس است، اما در RPC، داده ها باید از لایه های مختلف شبکه عبور کنند که این فرآیند می تواند صد ها هزار برابر کندتر باشد. این اختلاف مقیاس باعث می شود برنامه نویسی که به ماهیت توزیع شده سیستم آگاه نیست، با قرار دادن فراخوانی های RPC در حلقه های تکرار یا فراخوانی های زنجیره ای، گلوگاه های شدیدی در عملکرد سیستم ایجاد کند که در محیط تست محلی اصلاً به چشم نمی آیند، اما در محیط واقعی به دلیل تأخیر شبکه، کل سیستم را به زانو در می آورند.

از سوی دیگر، مدیریت منابع در RPC برخلاف فراخوانی محلی، به سادگی استفاده از حافظه پشته نیست. در فراخوانی محلی، سیستم عامل مدیریت چرخه حیات و پاکسازی منابع را بر عهده دارد، اما در RPC، اتصالات شبکه، بافرها و نشست های فعال باید به صورت صریح مدیریت شوند. غفلت از این پیچیدگی و برخورد با RPC به عنوان یک تابع ساده، می تواند به نشست منابع یا انباشت اتصالات باز و در نهایت از کار افتادن سرویس منجر شود. در نهایت، این انتزاع باعث می شود برنامه نویس از ماهیت ناپایدار و غیرقطعی شبکه غافل شود، در حالی که در یک سیستم توزیع شده، پذیرش احتمال خطا و طراحی برای شکست نه یک گزینه، بلکه یک الزام حیاتی برای بقای سیستم است که این شباهت ظاهری، آن را به شدت در ذهن توسعه دهنده کمرنگ می کند.

3) چه خطاهایی در فراخوانی راه دور ممکن است رخ دهد که در فراخوانی محلی وجود ندارد؟

همانطور که در مقایسه LPC و RPC و همچنین در بررسی خطاهای رایج RPC دیدیم، در حالی که در فراخوانی های محلی، سیستم حافظه و پردازنده به صورت یکپارچه عمل می کنند و خطاهای سخت افزاری حاد معمولاً منجر به توقف کامل و قطعی برنامه می شوند، در RPC ما با دنیایی از خطاهای غیرقطعی مواجه هستیم که در کدنویسی محلی اساساً موضوعیت ندارند. مهم ترین دسته از این خطاها ناشی از عدم قطعیت در فضای شبکه است؛ برای مثال، مشکل قطعی موقت یا دائمی اتصال یکی از رایج ترین مواردی است که باعث می شود کلاینت هیچگاه پاسخی دریافت نکند و مجبور به پیاده سازی مکانیزم های پیچیده Timeout و تلاش مجدد شود. فراتر از قطع شدن، ما با پدیده Partial Failure مواجه هستیم که در آن سرور درخواست را دریافت کرده، پردازش را آغاز نموده، اما قبل از ارسال پاسخ نهایی دچار خطا شده یا کرش می کند. این سناریو که در فراخوانی محلی غیرممکن است، کلاینت را در وضعیتی ابهام آمیز قرار می دهد که مشخص نیست آیا درخواست واقعاً اجرا شده است یا خیر؛ وضعیتی که به راحتی می تواند منجر به ناسازگاری داده ها در سیستم شود.

علاوه بر خطاهای ارتباطی، مشکل گم شدن یا تغییر محتوای بسته ها در مسیر شبکه نیز مطرح است که در فراخوانی های محلی به دلیل امنیت Memory Bus رخ نمی دهد. علاوه بر این، تفاوت در نسخه های نرم افزاری بین کلاینت و سرور می تواند منجر به خطاهای بسیار پیچیده ای در لایه Serialization شود؛ بدین معنا که سرور ممکن است فرمت داده ای را دریافت کند که با ساختار فعلی اش سازگار نیست یا پارامتر های ارسالی با انتظارات کد سمت سرور همخوانی ندارد. همچنین، مسئله اختلاف زمانی (Clock Skew) میان کلاینت و سرور نیز می تواند باعث بروز خطاهای منطقی در سیستم های توزیع شده شود که مبتنی بر زمان هستند، مشکلی که در یک پروسه محلی به دلیل وجود یک Clock مرجع واحد برای کل سیستم، هرگز مشاهده نمی شود. مجموع این خطاها باعث می شود که برنامه نویسی سیستم های توزیع شده مجبور باشد همواره فرض کند هر ارتباط راه دوری مستعد شکست است و باید برای هر فراخوانی، مکانیزم های دفاعی و مدیریت خطا پیش بینی کند، در حالی که چنین رویکردی برای فراخوانی های محلی کاملاً غیرضروری و سربار محسوب می شود.

4) در چه شرایطی gRPC انتخاب بهتری از REST است؟

gRPC زمانی به گزینه ای برتر تبدیل می شود که کارایی، صلیبیت ساختاری و ارتباطات پیچیده اولویت اصلی سیستم باشند. در سناریو هایی که با معماری میکروسرویس های روبرو هستیم، gRPC به دلیل بهره گیری از پروتکل HTTP/2 و Binary Serialization توسط Protocol Buffers، عملکردی به مراتب بهینه تر از REST ارائه می دهد. در واقع، زمانی که حجم ترافیک بین سرویس ها بسیار بالاست، حذف سربار متنی JSON و تبدیل آن به پکت های فشرده باینری در gRPC، نه تنها پهنای باند شبکه را به شکل چشم گیری حفظ می کند، بلکه بار پردازشی CPU برای فرآیند Serialization و Deserialization را نیز به حداقل می رساند؛ این موضوع در زیرساخت های ابری که هزینه منابع مصرفی مستقیماً با سودآوری در ارتباط است، یک مزیت اقتصادی و فنی بزرگ محسوب می شود.

علاوه بر بحث کارایی، gRPC در محیط های چندزبانه که در آن سرویس های مختلف با زبان های برنامه نویسی گوناگون مانند Go، Python و Java نوشته شده اند، نظم و امنیت بسیار بالاتری فراهم می کند. برخلاف REST که ماهیتی منعطف و گاهی بیش از حد آزاد دارد، gRPC بر پایه یک قرارداد سخت گیرانه در فایل های proto بنا شده است. این رویکرد قراردادی تضمین می کند که کلاینت و سرور همواره بر سر نوع داده ها و متد ها توافق دارند و خطاهای زمان اجرا ناشی از تغییرات ساختار داده ها (که در JSON بسیار رایج است) به شدت کاهش می یابد. بنابراین، در پروژه های بزرگ که هماهنگی بین تیم های مختلف دشوار است، استفاده از gRPC به عنوان منبع واحد برای تعریف واسطه ها، انتخاب بسیار هوشمندانه تری نسبت به REST است.

همچنین، در مواردی که نیاز به ارتباطات بلادرنگ و تعاملی وجود دارد، قابلیت های پیشرفته gRPC در مدیریت جریان داده ها آن را به رقیبی بی بدیل تبدیل می کند. در حالی که REST اساساً بر پایه مدل ساده ای Request-Response طراحی شده، gRPC به طور بومی از Streaming یک طرفه و دوطرفه پشتیبانی می کند. این ویژگی برای سیستم هایی که نیاز به ارسال مداوم داده ها دارند، مانند داشبورد های پایش لحظه ای، سرویس های چت، یا سیستم های دریافت داده از حسگر های IoT، حیاتی است؛ چرا که به جای باز و بسته کردن مداوم اتصالات، یک کانال ارتباطی پایدار و دوطرفه بر روی یک اتصال واحد TCP ایجاد می شود. در نهایت، gRPC برای اپلیکیشن های موبایل و دستگاه های با توان پردازشی محدود نیز انتخاب بهتری است، زیرا مصرف باتری و منابع دستگاه را به دلیل سبکی پکت های ارسالی و سرعت بالای پردازش آن ها، بهینه

سازی می‌کند، مشروط بر اینکه دسترسی مستقیم به زیرساخت شبکه و عدم نیاز به پشتیبانی مستقیم مرورگر (بدون واسطه) فراهم باشد.

5) در چه شرایطی REST انتخاب ساده تر یا مناسب تر است؟

با وجود مزایای فنی gRPC در محیط های داخلی و توزیع شده (و مواردی که در سوال قبلی به آنها اشاره شد)، REST به دلیل سادگی ذاتی و انطباق کامل با زیرساخت های فعلی وب، همچنان در بسیاری از سناریو ها انتخاب منطقی تر و مناسب تر محسوب می شود. اصلی ترین برتری REST در دسترس پذیری همگانی و عدم نیاز به ابزارهای جانبی برای برقراری ارتباط است. از آنجایی که REST بر پایه پروتکل استاندارد HTTP/1.1 بنا شده و از فرمت متنی JSON استفاده می کند، توسط تمامی مرورگرهای وب و زبان های برنامه نویسی بدون نیاز به نصب کتابخانه های پیچیده یا تولید کد پشتیبانی می شود. این ویژگی باعث می شود که REST برای توسعه واسطه های برنامه نویسی عمومی که قرار است توسط توسعه دهندگان شخص ثالث با سطوح دانش متفاوت استفاده شوند، به گزینه ای بی رقیب تبدیل شود؛ چرا که هر برنامه نویسی می تواند تنها با یک مرورگر یا ابزارهای ساده ای مثل cURL، خروجی سرویس را مشاهده و آن را بررسی کند.

علاوه بر این، فرآیند Debugging و Monitoring در REST به مراتب ساده تر از پروتکل های باینری است. در REST، داده ها به صورت متن خوانا جابه جا می شوند و این یعنی توسعه دهندگان می توانند بدون نیاز به رمزگشایی های پیچیده، محتوای درخواست ها و پاسخ ها را در لاگ های سرور یا ابزارهای تحلیل شبکه مشاهده کنند. این شفافیت در زمان بروز خطاهای منطقی، سرعت رفع مشکل را به شدت افزایش می دهد. همچنین، اکوسیستم ابزارهای مستند سازی در REST، مانند Swagger یا OpenAPI، به بلوغ کاملی رسیده است که اجازه می دهد مستندات تعاملی و خودخوان برای کاربران نهایی ایجاد شود؛ مزیتی که در gRPC به دلیل ماهیت صلب و مبتنی بر فایل های proto. با این سطح از سادگی و کاربرپسندی در دسترس نیست.

یکی دیگر از حوزه هایی که REST در آن برتری قاطع دارد، بهره گیری از مکانیزم های حافظه پنهان (Caching) در لایه های مختلف شبکه است. از آنجایی که REST از متدهای استاندارد HTTP مانند GET به درستی استفاده می کند، زیرساخت های وب مثل CDN ها، پروکسی های معکوس و حتی مرورگرها می توانند پاسخ ها را به طور خودکار ذخیره کنند تا از ارسال درخواست های تکراری به سرور جلوگیری شود. این قابلیت در gRPC که تقریباً تمام درخواست ها را در قالب متد POST ارسال می کند، به سادگی قابل پیاده سازی نیست. در نهایت، برای پروژه های کوچک تا متوسط، یا استارت آپ هایی که نیاز به توسعه سریع دارند، پیچیدگی تنظیمات اولیه gRPC، مدیریت فایل های تعریف واسطه و چالش های مربوط به Load Balancing در HTTP/2، ممکن است سرباری بیش از منفعت داشته باشد؛ در چنین شرایطی، REST با منحنی یادگیری ملایم و انعطاف پذیری بالای خود، اجازه می دهد تیم ها با سرعت بیشتری محصول خود را به بازار عرضه کنند.